

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №5
«процедуры, функции, триггеры в PostgreSQL »
по дисциплине «Проектирование и реализация баз данных»

Обучающийся Аксянова Алина Рустамовна
Факультет прикладной информатики
Группа К3240
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2023
Преподаватель Говорова Марина Михайловна

Санкт-Петербург
2025/2026

СОДЕРЖАНИЕ

Цель работы.....	3
Практическое задание.....	4
Часть 1. Три процедуры.....	5
Процедура 1. Добавление нового пассажира.....	5
Процедура 2. Изменение статуса билета.....	6
Процедура 3. Добавление транзитной посадки.....	7
Часть 2. Триггеры.....	8
Триггер 1. Логирование добавления пассажира.....	9
Триггер 2. Проверка цены билета перед вставкой.....	10
Триггер 3. Проверка способа продажи билета.....	11
Триггер 4. Проверка транзитной посадки.....	12
Триггер 5. Проверка разных аэропортов вылета и прибытия.....	12
Триггер 6. Логирование изменения статуса билета.....	13
Триггер 7. Проверка медосмотра.....	14
Вывод.....	15

Цель работы

овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание

1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров. Допустимо создать авторские процедуры. (3 балла)
2. Создать триггеры для индивидуальной БД согласно варианту:
Вариант 2.1. 7 триггеров по ИЗ.

Часть 1. Три процедуры

Процедура 1. Добавление нового пассажира

В базе данных была создана процедура для добавления нового пассажира в таблицу passenger. Процедура принимает фамилию, имя, отчество и паспортные данные пассажира и автоматически добавляет новую запись в таблицу. Использование процедуры позволяет упростить процесс добавления данных и избежать ручного написания SQL-запросов.

```
air_traffic_management=# CREATE OR REPLACE PROCEDURE airport_schema.add_passenger(
air_traffic_management(#      IN p_last_name varchar,
air_traffic_management(#      IN p_first_name varchar,
air_traffic_management(#      IN p_middle_name varchar,
air_traffic_management(#      IN p_passport_data varchar
air_traffic_management(# )
air_traffic_management=# LANGUAGE plpgsql
air_traffic_management=# AS $$
air_traffic_management=# BEGIN
air_traffic_management=#      INSERT INTO airport_schema.passenger
air_traffic_management=#      (last_name, first_name, middle_name, passport_data)
air_traffic_management=#      VALUES
air_traffic_management=#      (p_last_name, p_first_name, p_middle_name, p_passport_data);
air_traffic_management=# END;
air_traffic_management=# $$;
CREATE PROCEDURE
air_traffic_management=# CALL airport_schema.add_passenger(
air_traffic_management(#      'Сидоров',
air_traffic_management(#      'Артем',
air_traffic_management(#      'Викторович',
air_traffic_management(#      '5010 222222'
air_traffic_management(# );
CALL
air_traffic_management=#
air_traffic_management=# SELECT * FROM airport_schema.passenger;
passenger_id | last_name | first_name | middle_name | passport_data
-----
1 | Петров   | Егор      | Андреевич   | 5001 111111
2 | Орлова   | Светлана  | Ивановна    | 5002 111112
3 | Морозов  | Кирилл    | Олегович    | 5003 111113
4 | Васильева | Дарья     | Сергеевна   | 5004 111114
5 | Никитин  | Роман     | Павлович    | 5005 111115
6 | Сидоров  | Артем     | Викторович  | 5010 222222
(6 строк)
```

Процедура 2. Изменение статуса билета

Была создана процедура для изменения статуса билета. В качестве параметров передаются номер билета и новый статус. После выполнения процедуры статус билета обновляется в таблице ticket. Процедура используется для автоматизации изменения состояния билетов, например при регистрации пассажира или отмене рейса.

```
air_traffic_management=# CREATE OR REPLACE PROCEDURE airport_schema.update_ticket_status(
air_traffic_management(#      IN p_ticket_number varchar,
air_traffic_management(#      IN p_new_status varchar
air_traffic_management(# )
air_traffic_management=# LANGUAGE plpgsql
air_traffic_management=# AS $$
air_traffic_management$$ BEGIN
air_traffic_management$$      UPDATE airport_schema.ticket
air_traffic_management$$      SET ticket_status = p_new_status
air_traffic_management$$      WHERE ticket_number = p_ticket_number;
air_traffic_management$$ END;
air_traffic_management$$ $$;
CREATE PROCEDURE
air_traffic_management=# CALL airport_schema.update_ticket_status('T100001', 'checked_in');
CALL
air_traffic_management=#
air_traffic_management=# SELECT ticket_number, ticket_status
air_traffic_management=# FROM airport_schema.ticket
air_traffic_management=# WHERE ticket_number = 'T100001';
  ticket_number | ticket_status
-----+-----
  T100001      | checked_in
(1 строка)
```

Процедура 3. Добавление транзитной посадки

Для таблицы transit_stop была создана процедура добавления транзитной посадки. Процедура принимает данные о рейсе, аэропорте остановки, времени прибытия и вылета, а также времени стоянки. После выполнения процедуры новая запись автоматически добавляется в таблицу.

```
air_traffic_management=# CREATE OR REPLACE PROCEDURE airport_schema.add_transit_stop(
air_traffic_management(#      IN p_flight_id integer,
air_traffic_management(#      IN p_stop_order integer,
air_traffic_management(#      IN p_airport_id integer,
air_traffic_management(#      IN p_arrival_time time,
air_traffic_management(#      IN p_departure_time time,
air_traffic_management(#      IN p_ground_time_minutes integer
air_traffic_management(# )
air_traffic_management=# LANGUAGE plpgsql
air_traffic_management=# AS $$
air_traffic_management$$ BEGIN
air_traffic_management$$      INSERT INTO airport_schema.transit_stop
air_traffic_management$$      (flight_id, stop_order, airport_id, arrival_time, departure_time, ground_time_minutes)
air_traffic_management$$      VALUES
air_traffic_management$$      (p_flight_id, p_stop_order, p_airport_id, p_arrival_time, p_departure_time, p_ground_time_m
inutes);
air_traffic_management$$ END;
air_traffic_management$$ $$;
CREATE PROCEDURE
air_traffic_management=# CALL airport_schema.add_transit_stop(
air_traffic_management(#      1,
air_traffic_management(#      1,
air_traffic_management(#      3,
air_traffic_management(#      '10:00',
air_traffic_management(#      '10:40',
air_traffic_management(#      40
air_traffic_management(# );
CALL
air_traffic_management=#
air_traffic_management=# SELECT * FROM airport_schema.transit_stop;
transit_stop_id | flight_id | stop_order | airport_id | arrival_time | departure_time | ground_time_minutes
-----+-----+-----+-----+-----+-----+-----
1 | 4 | 1 | 2 | 16:20:00 | 17:00:00 | 40
2 | 4 | 2 | 4 | 20:30:00 | 21:10:00 | 40
3 | 1 | 1 | 3 | 10:00:00 | 10:40:00 | 40
(3 строки)
```

Часть 2. Триггеры

Таблица логов

Для хранения информации о действиях пользователей была создана таблица `action_log`. В таблице сохраняются название таблицы, тип выполненной операции, описание действия и время выполнения операции. Таблица используется для ведения истории изменений в базе данных.

```
air_traffic_management=# CREATE TABLE IF NOT EXISTS airport_schema.action_log (  
air_traffic_management(#   log_id serial PRIMARY KEY,  
air_traffic_management(#   table_name varchar(100) NOT NULL,  
air_traffic_management(#   operation varchar(20) NOT NULL,  
air_traffic_management(#   description text NOT NULL,  
air_traffic_management(#   log_time timestamp NOT NULL DEFAULT now()  
air_traffic_management(# );  
CREATE TABLE
```


Триггер 1. Логирование добавления пассажира

Был создан триггер, который автоматически записывает информацию в таблицу логов после добавления нового пассажира. При выполнении операции INSERT в таблицу passenger в таблицу action_log добавляется запись о создании нового пассажира.

```
air_traffic_management=# CREATE TABLE IF NOT EXISTS airport_schema.action_log (  
air_traffic_management(#   log_id serial PRIMARY KEY,  
air_traffic_management(#   table_name varchar(100) NOT NULL,  
air_traffic_management(#   operation varchar(20) NOT NULL,  
air_traffic_management(#   description text NOT NULL,  
air_traffic_management(#   log_time timestamp NOT NULL DEFAULT now()  
air_traffic_management(# );  
CREATE TABLE  
air_traffic_management=# CREATE OR REPLACE FUNCTION airport_schema.fn_log_passenger_insert()  
air_traffic_management=# RETURNS trigger AS $$  
air_traffic_management=$$ BEGIN  
air_traffic_management=#   INSERT INTO airport_schema.action_log(table_name, operation, description)  
air_traffic_management=#   VALUES (  
air_traffic_management=#     'passenger',  
air_traffic_management=#     'INSERT',  
air_traffic_management=#     'Добавлен пассажир: ' || NEW.last_name || ' ' || NEW.first_name  
air_traffic_management=#   );  
air_traffic_management=#   RETURN NEW;  
air_traffic_management=# END;  
air_traffic_management=# $$ LANGUAGE plpgsql;  
CREATE FUNCTION  
air_traffic_management=# DROP TRIGGER IF EXISTS trg_log_passenger_insert ON airport_schema.passenger;  
ЗАМЕЧАНИЕ: триггер "trg_log_passenger_insert" для отношения "airport_schema.passenger" не существует, пропускается  
DROP TRIGGER  
air_traffic_management=#  
air_traffic_management=# CREATE TRIGGER trg_log_passenger_insert  
air_traffic_management=# AFTER INSERT ON airport_schema.passenger  
air_traffic_management=# FOR EACH ROW  
air_traffic_management=# EXECUTE FUNCTION airport_schema.fn_log_passenger_insert();  
CREATE TRIGGER  
air_traffic_management=# INSERT INTO airport_schema.passenger  
air_traffic_management=# (last_name, first_name, middle_name, passport_data)  
air_traffic_management=# VALUES ('Федоров', 'Иван', 'Павлович', '5099 111111');  
INSERT 0 1  
air_traffic_management=#  
air_traffic_management=# SELECT * FROM airport_schema.action_log;  
  log_id | table_name | operation |      description      |      log_time  
-----+-----+-----+-----+-----  
      1 | passenger | INSERT   | Добавлен пассажир: Федоров Иван | 2026-05-15 14:03:20.729564  
(1 строка)
```

Триггер 2. Проверка цены билета перед вставкой

Создан триггер проверки стоимости билета перед вставкой или обновлением данных. Триггер запрещает добавление билетов с отрицательной стоимостью или отрицательной дополнительной платой. Это позволяет обеспечить корректность финансовых данных в базе.

```
air_traffic_management=# CREATE OR REPLACE FUNCTION airport_schema.fn_check_ticket_price()
air_traffic_management=# RETURNS trigger AS $$
air_traffic_management$# BEGIN
air_traffic_management$#     IF NEW.base_price <= 0 THEN
air_traffic_management$#         RAISE EXCEPTION 'Цена билета должна быть больше 0';
air_traffic_management$#     END IF;
air_traffic_management$#
air_traffic_management$#     IF NEW.extra_fee < 0 THEN
air_traffic_management$#         RAISE EXCEPTION 'Дополнительная плата не может быть отрицательной';
air_traffic_management$#     END IF;
air_traffic_management$#
air_traffic_management$#     RETURN NEW;
air_traffic_management$# END;
air_traffic_management$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
air_traffic_management=# DROP TRIGGER IF EXISTS trg_check_ticket_price ON airport_schema.ticket;
ЗАМЕЧАНИЕ: триггер "trg_check_ticket_price" для отношения "airport_schema.ticket" не существует, пропускается
DROP TRIGGER
air_traffic_management=#
air_traffic_management=# CREATE TRIGGER trg_check_ticket_price
air_traffic_management=# BEFORE INSERT OR UPDATE ON airport_schema.ticket
air_traffic_management=# FOR EACH ROW
air_traffic_management=# EXECUTE FUNCTION airport_schema.fn_check_ticket_price();
CREATE TRIGGER
air_traffic_management=# INSERT INTO airport_schema.ticket
air_traffic_management=# (ticket_number, flight_id, passenger_id, seat_number, seat_type, base_price, extra_fee, purchas
e_date, sale_type, cash_desk_id, ticket_status)
air_traffic_management=# VALUES
air_traffic_management=# ('BAD001', 1, 1, '10A', 'economy', -1000, 0, CURRENT_DATE, 'online', NULL, 'paid');
ОШИБКА: Цена билета должна быть больше 0
КОНТЕКСТ: функция PL/pgSQL airport_schema.fn_check_ticket_price(), строка 4, оператор RAISE
air_traffic_management=#
```

Триггер 3. Проверка способа продажи билета

Для таблицы ticket был создан триггер, проверяющий корректность способа продажи билета. Если билет продается онлайн, касса не должна указываться. Если продажа осуществляется через кассу, поле cash_desk_id должно быть заполнено. При нарушении условий операция блокируется.

```
air_traffic_management=# CREATE OR REPLACE FUNCTION airport_schema.fn_check_ticket_sale_type()
air_traffic_management=# RETURNS trigger AS $$
air_traffic_management=# BEGIN
air_traffic_management=#     IF NEW.sale_type = 'online' AND NEW.cash_desk_id IS NOT NULL THEN
air_traffic_management=#         RAISE EXCEPTION 'Для онлайн-продажи касса не указывается';
air_traffic_management=#     END IF;
air_traffic_management=#
air_traffic_management=#     IF NEW.sale_type = 'desk' AND NEW.cash_desk_id IS NULL THEN
air_traffic_management=#         RAISE EXCEPTION 'Для продажи через кассу необходимо указать cash_desk_id';
air_traffic_management=#     END IF;
air_traffic_management=#
air_traffic_management=#     RETURN NEW;
air_traffic_management=# END;
air_traffic_management=# $$ LANGUAGE plpgsql;
CREATE FUNCTION
air_traffic_management=# DROP TRIGGER IF EXISTS trg_check_ticket_sale_type ON airport_schema.ticket;
ЗАМЕЧАНИЕ: триггер "trg_check_ticket_sale_type" для отношения "airport_schema.ticket" не существует, пропускается
DROP TRIGGER
air_traffic_management=#
air_traffic_management=# CREATE TRIGGER trg_check_ticket_sale_type
air_traffic_management=# BEFORE INSERT OR UPDATE ON airport_schema.ticket
air_traffic_management=# FOR EACH ROW
air_traffic_management=# EXECUTE FUNCTION airport_schema.fn_check_ticket_sale_type();
CREATE TRIGGER
air_traffic_management=# INSERT INTO airport_schema.ticket
air_traffic_management=# (ticket_number, flight_id, passenger_id, seat_number, seat_type, base_price, extra_fee, purchas
e_date, sale_type, cash_desk_id, ticket_status)
air_traffic_management=# VALUES
air_traffic_management=# ('BAD002', 1, 1, '10B', 'economy', 9000, 0, CURRENT_DATE, 'desk', NULL, 'paid');
ОШИБКА: Для продажи через кассу необходимо указать cash_desk_id
КОНТЕКСТ: функция PL/pgSQL airport_schema.fn_check_ticket_sale_type(), строка 8, оператор RAISE
air_traffic_management=# |
```

Триггер 4. Проверка транзитной посадки

Создан триггер проверки данных транзитной посадки. Триггер контролирует, чтобы номер остановки находился в допустимом диапазоне, а время вылета было позже времени прибытия. Это позволяет избежать ошибок в расписании рейсов.

```
air_traffic_management=# CREATE OR REPLACE FUNCTION airport_schema.fn_check_transit_stop()
air_traffic_management=# RETURNS trigger AS $$
air_traffic_management$# BEGIN
air_traffic_management$#     IF NEW.stop_order < 1 OR NEW.stop_order > 3 THEN
air_traffic_management$#         RAISE EXCEPTION 'Порядковый номер транзитной посадки должен быть от 1 до 3';
air_traffic_management$#     END IF;
air_traffic_management$#
air_traffic_management$#     IF NEW.departure_time <= NEW.arrival_time THEN
air_traffic_management$#         RAISE EXCEPTION 'Время вылета из транзитного аэропорта должно быть позже времени прилет
a';
air_traffic_management$#     END IF;
air_traffic_management$#
air_traffic_management$#     RETURN NEW;
air_traffic_management$# END;
air_traffic_management$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
air_traffic_management=# DROP TRIGGER IF EXISTS trg_check_transit_stop ON airport_schema.transit_stop;
ЗАМЕЧАНИЕ: триггер "trg_check_transit_stop" для отношения "airport_schema.transit_stop" не существует, пропускается
DROP TRIGGER
air_traffic_management=#
air_traffic_management=# CREATE TRIGGER trg_check_transit_stop
air_traffic_management=# BEFORE INSERT OR UPDATE ON airport_schema.transit_stop
air_traffic_management=# FOR EACH ROW
air_traffic_management=# EXECUTE FUNCTION airport_schema.fn_check_transit_stop();
CREATE TRIGGER
air_traffic_management=# INSERT INTO airport_schema.transit_stop
air_traffic_management=# (flight_id, stop_order, airport_id, arrival_time, departure_time, ground_time_minutes)
air_traffic_management=# VALUES
air_traffic_management=# (1, 4, 2, '12:00', '12:30', 30);
ОШИБКА: Порядковый номер транзитной посадки должен быть от 1 до 3
КОНТЕКСТ: функция PL/pgSQL airport_schema.fn_check_transit_stop(), строка 4, оператор RAISE
air_traffic_management=# |
```

Триггер 5. Проверка разных аэропортов вылета и прибытия

Был создан триггер, запрещающий создание рейсов с одинаковыми аэропортами вылета и прибытия. При попытке добавить такой рейс PostgreSQL выдает ошибку и операция не выполняется.

```
air_traffic_management=# CREATE OR REPLACE FUNCTION airport_schema.fn_check_flight_airports()
air_traffic_management=# RETURNS trigger AS $$
air_traffic_management$# BEGIN
air_traffic_management$#     IF NEW.departure_airport_id = NEW.arrival_airport_id THEN
air_traffic_management$#         RAISE EXCEPTION 'Аэропорт вылета и аэропорт назначения не должны совпадать';
air_traffic_management$#     END IF;
air_traffic_management$#
air_traffic_management$#     RETURN NEW;
air_traffic_management$# END;
air_traffic_management$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
air_traffic_management=# DROP TRIGGER IF EXISTS trg_check_flight_airports ON airport_schema.flight;
ЗАМЕЧАНИЕ: триггер "trg_check_flight_airports" для отношения "airport_schema.flight" не существует, пропускается
DROP TRIGGER
air_traffic_management=#
air_traffic_management=# CREATE TRIGGER trg_check_flight_airports
air_traffic_management=# BEFORE INSERT OR UPDATE ON airport_schema.flight
air_traffic_management=# FOR EACH ROW
air_traffic_management=# EXECUTE FUNCTION airport_schema.fn_check_flight_airports();
CREATE TRIGGER
air_traffic_management=# INSERT INTO airport_schema.flight
air_traffic_management=# (flight_number, departure_date, departure_time, departure_airport_id, arrival_airport_id, aircr
aft_id, distance_km, flight_kind)
air_traffic_management=# VALUES
air_traffic_management=# ('BADFL1', CURRENT_DATE, '12:00', 1, 1, 1, 1000, 'scheduled');
ОШИБКА: Аэропорт вылета и аэропорт назначения не должны совпадать
КОНТЕКСТ: функция PL/pgSQL airport_schema.fn_check_flight_airports(), строка 4, оператор RAISE
air_traffic_management=# |
```

Триггер 6. Логирование изменения статуса билета

Создан триггер, автоматически записывающий изменение статуса билета в таблицу логов. После изменения статуса в журнал добавляется информация о старом и новом значении статуса.

```
air_traffic_management=# CREATE OR REPLACE FUNCTION airport_schema.fn_log_ticket_status_update()
air_traffic_management=# RETURNS trigger AS $$
air_traffic_management=# BEGIN
air_traffic_management=#     IF OLD.ticket_status <> NEW.ticket_status THEN
air_traffic_management=#         INSERT INTO airport_schema.action_log(table_name, operation, description)
air_traffic_management=#             VALUES (
air_traffic_management=#                 'ticket',
air_traffic_management=#                 'UPDATE',
air_traffic_management=#                 'Изменен статус билета ' || NEW.ticket_number ||
air_traffic_management=#                 ': ' || OLD.ticket_status || ' -> ' || NEW.ticket_status
air_traffic_management=#             );
air_traffic_management=#     END IF;
air_traffic_management=#     RETURN NEW;
air_traffic_management=# END;
air_traffic_management=# $$ LANGUAGE plpgsql;
CREATE FUNCTION
air_traffic_management=# DROP TRIGGER IF EXISTS trg_log_ticket_status_update ON airport_schema.ticket;
ЗАМЕЧАНИЕ: триггер "trg_log_ticket_status_update" для отношения "airport_schema.ticket" не существует, пропускается
DROP TRIGGER
air_traffic_management=#
air_traffic_management=# CREATE TRIGGER trg_log_ticket_status_update
air_traffic_management=# AFTER UPDATE ON airport_schema.ticket
air_traffic_management=# FOR EACH ROW
air_traffic_management=# EXECUTE FUNCTION airport_schema.fn_log_ticket_status_update();
CREATE TRIGGER
air_traffic_management=# UPDATE airport_schema.ticket
air_traffic_management=# SET ticket_status = 'checked_in'
air_traffic_management=# WHERE ticket_number = 'T100002';
UPDATE 1
air_traffic_management=#
air_traffic_management=# SELECT * FROM airport_schema.action_log;
 log_id | table_name | operation | description | log_time
-----+-----+-----+-----+-----
1 | passenger | INSERT | Добавлен пассажир: Федоров Иван | 2026-05-15 14:03:20.729564
2 | ticket | UPDATE | Изменен статус билета T100002: paid -> checked_in | 2026-05-15 14:17:46.692016
(2 строки)
```

Триггер 7. Проверка медосмотра

Для таблицы `medical_exam` был создан триггер проверки результатов медицинского осмотра сотрудников. Если сотрудник допущен к работе, причина недопуска должна отсутствовать. Если сотрудник не допущен, причина должна быть указана обязательно. Триггер обеспечивает корректность хранения медицинских данных.

```
air_traffic_management=# CREATE OR REPLACE FUNCTION airport_schema.fn_check_medical_exam()
air_traffic_management-# RETURNS trigger AS $$
air_traffic_management=# BEGIN
air_traffic_management$#     IF NEW.exam_status = 'allowed' AND NEW.rejection_reason IS NOT NULL THEN
air_traffic_management$#         RAISE EXCEPTION 'Для статуса allowed причина недопуска должна быть пустой';
air_traffic_management$#     END IF;
air_traffic_management$#
air_traffic_management$#     IF NEW.exam_status = 'not_allowed' AND NEW.rejection_reason IS NULL THEN
air_traffic_management$#         RAISE EXCEPTION 'Для статуса not_allowed необходимо указать причину недопуска';
air_traffic_management$#     END IF;
air_traffic_management$#
air_traffic_management$#     RETURN NEW;
air_traffic_management$# END;
air_traffic_management$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
air_traffic_management=# DROP TRIGGER IF EXISTS trg_check_medical_exam ON airport_schema.medical_exam;
ЗАМЕЧАНИЕ: триггер "trg_check_medical_exam" для отношения "airport_schema.medical_exam" не существует, пропускается
DROP TRIGGER
air_traffic_management=#
air_traffic_management=# CREATE TRIGGER trg_check_medical_exam
air_traffic_management-# BEFORE INSERT OR UPDATE ON airport_schema.medical_exam
air_traffic_management-# FOR EACH ROW
air_traffic_management-# EXECUTE FUNCTION airport_schema.fn_check_medical_exam();
CREATE TRIGGER
air_traffic_management=# INSERT INTO airport_schema.medical_exam
air_traffic_management-# (crew_assignment_id, exam_date, exam_status, rejection_reason)
air_traffic_management-# VALUES
air_traffic_management-# (1, CURRENT_DATE, 'not_allowed', NULL);
ОШИБКА: Для статуса not_allowed необходимо указать причину недопуска
КОНТЕКСТ: функция PL/pgSQL airport_schema.fn_check_medical_exam(), строка 8, оператор RAISE
air_traffic_management=# |
```

Вывод

В ходе лабораторной работы были изучены механизмы создания процедур, функций и триггеров в PostgreSQL.

Были реализованы: процедуры для автоматизации работы с данными, триггеры для контроля целостности информации, механизмы логирования действий пользователей, проверки корректности бизнес-логики базы данных.

В результате были получены практические навыки работы с PL/pgSQL, создания триггерных функций и организации автоматического контроля данных в реляционных базах данных.